

# 141. Ce que le compilateur refuse, et pourquoi

Le dernier tutoriel n'apprend pas à écrire : il apprend à lire les refus.

## 1. Ce qu'il faut comprendre

C'est la règle qui gouverne le projet. Un `long` traduit en douce sur un octet, c'est un jeu qui compte faux à partir de 256 et personne n'a été prévenu ; un refus, c'est trois secondes perdues et une ligne à réécrire.

Cinq refus qu'on rencontre vraiment. `float vitesse = 1.5;` — la machine n'a pas de virgule. `class Ennemi { ... };` — le message propose `struct`, qui fait ce qu'on voulait, méthodes comprises. `carte[3] = 1;` sur une grille — il faut **deux** index, car `carte[3]` désigne une rangée entière. `uint8_t loin(Ennemi e)` — un argument est un octet ; une `struct` se passe par son nom, visible partout, ou par son index. `TABLE[0] = 9;` sur une table `const` — elle est gravée dans la cartouche.

Chacun porte **son numéro de ligne**, et dit quoi écrire à la place. C'est ce qui fait la différence entre un compilateur qui aide et un compilateur qui punit : les deux disent non, un seul dit pourquoi. Le programme ci-dessous est la version acceptée des cinq.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **763 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
loin()          28 o ← nombre() - ta fonction
main()         113 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
EcrireTexte     10 o ← texte() - écrit un texte, lettre par lettre
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    94 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         336 o ← CopierTuiles - 21 lettres sur 43
"ECART"         5 o ← texte() - un texte gravé
"GRILLE"        6 o ← texte() - un texte gravé
"TABLE"         5 o ← texte() - un texte gravé
"PAS"           3 o ← texte() - un texte gravé
TABLE_0         3 o ← nombre() - une table gravée
```

Lettres gravées : ABCEGILPRST0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte()  × 4    44 o sur place, dans main() + EcrireTexte (10 o)
nombre() × 4    41 o sur place, dans main() + loin() (28 o), EcrireNombre (33 o), TABLE_0 (3 o)
image()  × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
loin()   × 1     7 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : la version ACCEPTÉE de cinq lignes que le compilateur refuse.
// Il affiche quatre valeurs : un écart, une case de grille, une case de table, une vitesse.
//
// Les cinq refus, et ce qu'on écrit à la place :
// 1. un nombre à virgule (float)      → un entier uint8_t (pas de virgule ici)
// 2. class                            → struct
// 3. une grille avec un seul index    → deux index : carte[2][3]
// 4. une struct donnée à une fonction → son numéro, ou la struct par son nom
// 5. changer une table const          → impossible : elle est gravée

#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
```

## 2. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi { uint8_t x, y; };           // struct (et pas class)
```

## 3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — TABLE

```
const uint8_t TABLE[] = { 1, 2, 3 };     // une table gravée : on la lit, on ne l'écrit pas
```

## 4. UN TABLEAU — CARTE

```
uint8_t carte[4][5];                      // une grille : 4 lignes de 5 cases
```

## 5. UNE VARIABLE — BOSS

```
Ennemi boss;                             // un ennemi, visible partout

// Pas la struct : son INDEX. Un argument est un octet.
```

## 6. UNE FONCTION — LOIN()

```
uint8_t loin(uint8_t cible) {              // Rend l'écart entre boss.x et cible.
    if (boss.x > cible) return boss.x - cible;
    return cible - boss.x;
}
```

## 7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    boss.x = 40;
    carte[2][3] = 1;                        // deux index, pas un

    uint8_t vitesse = 2;                    // un octet, pas un float

    texte(1, 4, "ECART");
    nombre(9, 4, loin(10));                 // 40 - 10 = 30 : affiche 030

    texte(1, 6, "GRILLE");
    nombre(9, 6, carte[2][3], 1);           // 1

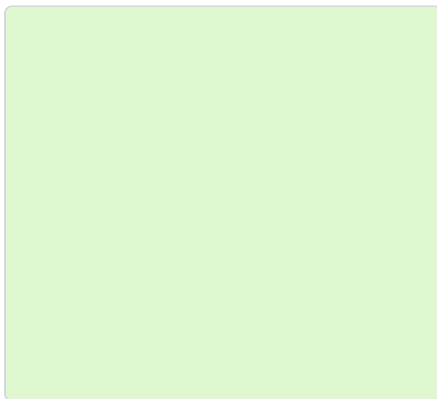
    texte(1, 8, "TABLE");
    nombre(9, 8, TABLE[2], 1);             // la case 2 de la table : 3

    texte(1, 10, "PAS");
    nombre(9, 10, vitesse, 1);              // 2

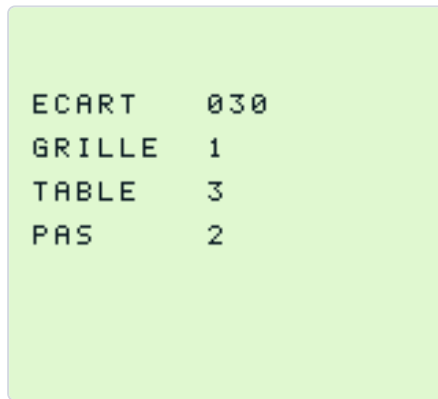
    while (true) {                          // Répète pour toujours :
        image();                            // attend l'image suivante.
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

030, 1, 3, 2 — les cinq refus, écrits comme il faut.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la struct passe par son index

✓ la grille prend ses deux index

✓ la table gravée se lit

✓ et l'octet remplace le float

---

Ensuite : **142. Ce qu'une ligne de C++ devient, en vrai** — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche Game Boy en C++